💻 Development Ticket: TKT-8042

Feature: Fintech Transaction Engine & Account Models

Context

We are building the core transaction backend for a new digital bank. The system needs to securely handle customer funds, support specialized corporate clients, and process dynamic fee structures. Your task is to design this system by applying core Object-Oriented design principles to meet the strict business rules outlined below.

1. Core Data Security & Validation

- Create a base blueprint named `BankAccount`.
- It must hold two data points: `accountNumber` (String) and `balance` (double).
- **Security Rule:** These data points must be strictly protected from direct outside access. Other parts of the application should only be able to read or modify them through controlled access methods (getters and setters).
- **Business Rule:** The bank has a strict no-overdraft policy. Inside your modification method (setter) for the balance, write logic to inspect the incoming value. If the value is less than 0, reject the operation, print *"Alert: Invalid operation. Balance cannot be negative."* and leave the balance at 0.0.

2. Account Specialization

- The bank is expanding to support B2B clients. Create a new class named `BusinessAccount`.
- Instead of rewriting all the code from scratch, this new class must absorb and build upon all the properties and behaviors of the standard `BankAccount`.
- Add a new, specific property to this specialized class: `companyTaxId` (String).

3. Dynamic Fee Calculation

- In the base `BankAccount` class, create a behavior called `calculateTransferFee` that accepts a `transferAmount` (double). For standard accounts, this should simply return a flat fee of **$5.00** regardless of the amount.

- Corporate clients have a different contract. In your `BusinessAccount` class, provide a custom implementation of the exact same `calculateTransferFee` method. For business accounts, this behavior must replace the flat fee and instead return **1.5%** of the `transferAmount`.

4. Flexible Reporting

- In the `BankAccount` class, create a behavior named `generateStatement`. It should take no parameters and simply print the account number and current balance.
- The business now wants to generate statements in different file types, but they want developers to use the exact same action name. Create a **second variation** of the `generateStatement` method. It must have the exact same method name, but this version will accept a `formatType` (String) parameter.
- In this second variation, if "PDF" is passed, print *"Generating secure PDF..."*. If "CSV" is passed, print *"Exporting CSV spreadsheet..."*.

System Testing (The Main Class)

Create a `Main` execution class to prove your architecture works. **(Do not use arrays or collections)**:

1. Instantiate exactly one standard account and one business account.
2. Use your secure access methods to assign data. Purposely try to set the standard account balance to a negative number to prove your security validation blocks it.
3. Call `calculateTransferFee(1000)` on both objects and print the results to prove the system automatically knows which calculation to use (one should charge $5.0, the other should charge $15.0).
4. Call both variations of `generateStatement` to prove your flexible reporting works.